COSC 42: Distributed Computing

Problem 1. Model Checking Peterson's Algorithm.
Specify Peterson's Algorithm in TLA+. Formulate Mutual Exclusion of Peterson's Algorithm as an invariant. Use the TLC Model Checker to prove (by exhaustive search) that Peterson's Algorithm satisfies Mutual Exclusion.

Solution.

TLC exhaustively found all 66 (32 distinct) reachable states and found no violation of Mutex. Thus, the model satisfies mutual exclusion.

TLA+ model

MODULE *problem*1

EXTENDS TLC, *Integers*

VARIABLES *interested*, *pc*, *turn*

$processes \triangleq \{0, 1\}$

$TypeOK \triangleq$
$\quad \wedge \quad interested \in [processes \to \text{BOOLEAN}\ ]$
$\quad \wedge \quad turn \in processes$
$\quad \wedge \quad pc \in [processes \to \{\text{“T1”}, \text{“T2”}, \text{“T3A”}, \text{“T3B”}, \text{“E1”}\}]$

$Init \triangleq$
$\quad \wedge\ interested = [i \in processes \mapsto \text{FALSE}]$
$\quad \wedge\ turn \in processes$
$\quad \wedge\ pc = [p \in processes \mapsto \text{“T1”}]$

$T1(p) \triangleq$
$\quad \wedge\ pc[p] = \text{“T1”}$
$\quad \wedge\ pc' = [pc \text{ EXCEPT } ![p] = \text{“T2”}]$
$\quad \wedge\ interested' = [interested \text{ EXCEPT } ![p] = \text{TRUE}]$
$\quad \wedge\ \text{UNCHANGED } turn$

$T2(p) \triangleq$
$\quad \wedge\ pc[p] = \text{“T2”}$
$\quad \wedge\ pc' = [pc \text{ EXCEPT } ![p] = \text{“T3A”}]$
$\quad \wedge\ turn' = 1 - p$
$\quad \wedge\ \text{UNCHANGED } interested$

TLA+ model continued

$$
\begin{array}{l}
T3A(p) \triangleq \\
\quad \land pc[p] = \text{“T3A”} \\
\quad \land pc' = [pc \text{ EXCEPT } ![p] = \\
\qquad \text{IF } turn = p \text{ THEN “E1”} \\
\qquad \text{ELSE “T3B”}] \\
\quad \land \text{UNCHANGED } \langle turn, interested \rangle \\
\\
T3B(p) \triangleq \\
\quad \land pc[p] = \text{“T3B”} \\
\quad \land pc' = [pc \text{ EXCEPT } ![p] = \\
\qquad \text{IF } interested[1-p] \text{ THEN “T3A”} \\
\qquad \text{ELSE “E1”}] \\
\quad \land \text{UNCHANGED } \langle turn, interested \rangle \\
\\
E1(p) \triangleq \\
\quad \land pc[p] = \text{“E1”} \\
\quad \land pc' = [pc \text{ EXCEPT } ![p] = \text{“T1”}] \\
\quad \land interested' = [interested \text{ EXCEPT } ![p] = \text{FALSE}] \\
\quad \land \text{UNCHANGED } turn \\
\\
Mutex \triangleq \\
\quad \neg(pc[1] = \text{“E1”} \land pc[0] = \text{“E1”}) \\
\\
Next \triangleq \\
\quad \exists\, p \in processes : \\
\qquad \lor T1(p) \\
\qquad \lor T2(p) \\
\qquad \lor T3A(p) \\
\qquad \lor T3B(p) \\
\qquad \lor E1(p)
\end{array}
$$

TLC configuration

```
INIT Init
NEXT Next
INVARIANTS
    TypeOK
    Mutex
```

Problem 2. Puli Meka Gaddi.
The following is a Telugu folk puzzle called *Puli Meka Gaddi* (translating to *Tiger, Goat, and Hay*).
A boatman has a tiger, a goat, and a bale of hay and is on the left bank of a river. His objective is to cross from the left bank of the river to the right bank of the river and to end up on the right bank of the river with all of his three life-forms (tiger, goat, and hay). Unfortunately, his boat only has space for him and one of the three life-forms, therefore he can take at most one of the life-forms with him in the boat at a time. Thus, he must necessarily make several left-to-right and right-to-left trips across the river to achieve his goal. In addition, the boatman also faces the following conundrum, if he leaves the tiger and goat alone together (without him), the tiger will eat the goat. Similarly, if he leaves the goat and hay alone together, the goat will eat the hay. The Puli Meka Gaddi Puzzle is to find a way for the boatman to successfully end up on the right bank of the river along with the tiger, the goat, and the hay without anything getting eaten.
In this problem, your objective is not just to solve the Puli Meka Gaddi puzzle, but to actually make the TLC model checker solve the puzzle for you. In particular, recall that the TLC model checker can check for invariants in TLA+ model specifications. Can you figure out how to use the TLC model checker to solve Puli Meka Gaddi for you?
In particular, you should specify the Puli Meka Gaddi system as a TLA+ model and formulate an appropriate non-invariant, such that model checking your non-invariant in TLC will lead the model checker to throw an "error" trace that is a solution to the Puli Meka Gaddi Puzzle. Submit your pretty-printed TLA+ model and the solution returned by TLC, along with a translation of the solution returned by TLC into plain English.

Solution.

TLA+ model

MODULE *problem3*

EXTENDS *TLC*, *FiniteSets*, *Integers*

VARIABLES *boatman*, *left*, *right*

$Items \triangleq \{\text{“tiger”}, \text{“goat”}, \text{“hay”}\}$

$Init \triangleq$
- $\land\ boatman = \text{“left”}$
- $\land\ left = Items$
- $\land\ right = \{\}$

$TypeOK \triangleq$
- $\land\ \ boatman \in \{\text{“left”}, \text{“right”}\}$
- $\land\ \ left \in \text{SUBSET } Items$
- $\land\ \ right \in \text{SUBSET } Items$

$BanksOK \triangleq$
- $\land\ left \cap right = \{\}$
- $\land\ left \cup right = Items$

$Dangerous(bank) \triangleq$
- $\lor\ \{\text{“tiger”}, \text{“goat”}\} \subseteq bank$
- $\lor\ \{\text{“hay”}, \text{“goat”}\} \subseteq bank$

$SafeBank(bank, side) \triangleq$
- $\neg(Dangerous(bank) \land boatman \neq side)$

$Safe \triangleq$
- $\land\ SafeBank(left, \text{“left”})$
- $\land\ SafeBank(right, \text{“right”})$

$Cross(start, start_side, end, end_side, cargo) \triangleq$
- $\land\ boatman = start_side$
- $\land\ boatman' = end_side$
- $\land\ cargo \subseteq start$
- $\land\ Cardinality(cargo) \leq 1$
- $\land\ start' = start \setminus cargo$
- $\land\ end' = end \cup cargo$
- $\land\ Safe'$

$Next \triangleq$
- $\lor\ \exists\, cargo \in \text{SUBSET } left :$
  - $Cross(left, \text{“left”}, right, \text{“right”}, cargo)$
- $\lor\ \exists\, cargo \in \text{SUBSET } right :$
  - $Cross(right, \text{“right”}, left, \text{“left”}, cargo)$

$Goal \triangleq$
- $\land\ boatman = \text{“right”}$
- $\land\ right = Items$
- $\land\ left = \{\}$

$NotGoal \triangleq \neg Goal$

TLC configuration

```
INIT Init
NEXT  Next
INVARIANTS
    BanksOK
    TypeOK
    Safe
    NotGoal
```

Solution returned by TLC:

Solution returned by TLC

```
State 1: Initial state
/\ boatman = "left"
/\ right = {}
/\ left = {"tiger", "goat", "hay"}

State 2
/\ boatman = "right"
/\ right = {"goat"}
/\ left = {"tiger", "hay"}

State 3
/\ boatman = "left"
/\ right = {"goat"}
/\ left = {"tiger", "hay"}

State 4
/\ boatman = "right"
/\ right = {"tiger", "goat"}
/\ left = {"hay"}

State 5
/\ boatman = "left"
/\ right = {"tiger"}
/\ left = {"goat", "hay"}

State 6
/\ boatman = "right"
/\ right = {"tiger", "hay"}
```

```
/\ left = {"goat"}

State 7
/\ boatman = "left"
/\ right = {"tiger", "hay"}
/\ left = {"goat"}

State 8
/\ boatman = "right"
/\ right = {"tiger", "goat", "hay"}
/\ left = {}
```

English explanation:

We can solve the puzzle by following the following steps:

- Step 0: They all start on the left bank of the river.
- Step 1: The boatman crosses to the right bank with the goat, drops it there and goes back to the left bank.
- Step 2: The boatman crosses to the right bank with the tiger, drops it there, and brings the goat back to the left bank.
- Step 3: The boatman crosses to the right bank with the hay, drops it there, and goes back to the left bank.
- Step 4: The boatman crosses to the right bank with the goat.

Thus successfully ending up on the right bank of the river along with the tiger, the goat, and the hay without anything getting eaten.